# AI Resume Analyzer & Job Matcher

B.Tech CSE Final Year Project — Complete Architecture, Source Code & Setup Guide

This project implements an Automated Applicant Tracking System (ATS) style Resume Analyzer and Job Matcher using Natural Language Processing (NLP). It parses uploaded PDF resumes, evaluates similarity against target job descriptions, and highlights missing technical keywords.

## 1. System Architecture & Tech Stack

- **Frontend Interface:** Streamlit (enables rapid deployment of an interactive Python web application).
- **Backend & NLP Engine:** Python, `PyPDF2` (PDF text extraction), `NLTK` (text preprocessing and stopword filtering), and `Scikit-Learn` (TF-IDF vectorization and cosine similarity scoring).
- **Upgrade Path:** Once the baseline prototype works, TF-IDF can be replaced with HuggingFace `Sentence-Transformers` (`all-MiniLM-L6-v2`) for deep semantic matching.

## 2. Project Folder Structure

Create a root project folder named `ai-resume-matcher` and organize the files as follows:

```
ai-resume-matcher/
├── app.py               # Main Streamlit web interface
├── analyzer.py          # NLP text extraction and matching engine
├── requirements.txt     # Python package dependencies
└── README.md            # Project documentation for GitHub
```

## 3. Complete Source Code

### Step A: requirements.txt

Create this file to specify the required Python libraries and versions:

```
streamlit==1.38.0
PyPDF2==3.0.1
scikit-learn==1.5.1
nltk==3.9.1
```

### Step B: analyzer.py

This module handles PDF text extraction, regex text cleaning, stopword removal, TF-IDF vectorization, cosine similarity calculation, and missing keyword extraction:

```
import re
import PyPDF2
import nltk
from nltk.corpus import stopwords
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

nltk.download('stopwords', quiet=True)
STOP_WORDS = set(stopwords.words('english'))

def extract_text_from_pdf(pdf_file):
    reader = PyPDF2.PdfReader(pdf_file)
    text = ""
    for page in reader.pages:
        extracted = page.extract_text()
        if extracted:
            text += extracted + " "
    return text

def clean_text(text):
    text = text.lower()
    text = re.sub(r'[^a-z0-9\s+#.]', ' ', text)
    words = [word for word in text.split() if word not in STOP_WORDS and len(word) >
1]
    return " ".join(words)

def calculate_match(resume_text, jd_text):
    cleaned_resume = clean_text(resume_text)
    cleaned_jd = clean_text(jd_text)

    vectorizer = TfidfVectorizer()
    vectors = vectorizer.fit_transform([cleaned_resume, cleaned_jd])
    score = cosine_similarity(vectors[0:1], vectors[1:2])[0][0] * 100

    resume_words = set(cleaned_resume.split())
    jd_words = set(cleaned_jd.split())
    missing_keywords = sorted(list(jd_words - resume_words))

    return round(score, 2), missing_keywords[:15]
```

### Step C: app.py

This script creates the interactive web interface where users upload their PDF resume and paste a target job description:

```
import streamlit as st
from analyzer import extract_text_from_pdf, calculate_match

st.set_page_config(page_title="AI Resume Analyzer", layout="centered")

st.title("AI Resume Analyzer & Job Matcher")
st.write("Upload your resume and paste the target job description to check your ATS
compatibility.")

uploaded_file = st.file_uploader("Upload Resume (PDF format)", type=["pdf"])
job_description = st.text_area("Paste Job Description Here", height=200)

if st.button("Analyze Resume"):
    if uploaded_file is not None and job_description.strip() != "":
        with st.spinner("Analyzing resume..."):
            resume_text = extract_text_from_pdf(uploaded_file)
            score, missing_keywords = calculate_match(resume_text, job_description)

        st.subheader(f"Match Score: {score}%")
        st.progress(int(score))

        if score >= 75:
            st.success("Strong match! Your resume aligns well with this role.")
        elif score >= 50:
            st.warning("Moderate match. Consider adding the missing keywords below.")
        else:
            st.error("Low match. Tailor your skills and project descriptions to fit
the role.")

        st.subheader("Top Missing Keywords from Job Description")
        if missing_keywords:
            st.write(", ".join(missing_keywords))
        else:
            st.write("Great job! No major keywords are missing.")
    else:
        st.error("Please upload a PDF resume and enter a job description.")
```

## 4. Local Setup & Execution Steps

1. **Open Terminal:** Navigate inside your `ai-resume-matcher` project folder.
2. **Install Dependencies:** Run `pip install -r requirements.txt` to install all packages.
3. **Start the Web Server:** Run `streamlit run app.py` in your terminal.
4. **Test the Application:** Open `http://localhost:8501` in your browser, upload any PDF resume, paste a job description, and click **Analyze Resume**.

### 5. How to Make It Final-Year Viva Ready

- **Semantic Embeddings:** Upgrade from `TfidfVectorizer` to HuggingFace's `SentenceTransformer` so the system understands context and synonyms (e.g., matching "ML" with "Machine Learning").
- **Database Integration:** Connect PostgreSQL or MongoDB to store user scan history and recruiter analytics dashboards.
- **Cloud Deployment:** Host the application on Streamlit Community Cloud or Render so you can demonstrate a live URL during evaluations.